

CS 2110 - Spring 2026

Project 4: C Programming

Andy Garcha, Yash Choudhury, Suraj Samudrala, William Sugden, Seha Park,
James Davis, Nishi Agrawal, Rohit Rao, Daniel Huang, Jason Tran,
Tanay Kapoor, Chris Goggins, Cole Saunders

Due: 4/10/2026 @ 11:59 PM
Version: 1.0

Contents

1	General	2
1.1	Introduction	2
1.2	What functions can I use?	3
2	Assembly Functions	4
2.1	Overview	4
2.2	Functions	4
2.2.1	Longest Wordle Streak	4
2.2.2	Valid Sudoku Row	4
2.2.3	Create Letter Boxed	5
2.2.4	Wordle Score	5
2.2.5	Valid Spelling Bee	5
2.2.6	Octal String to Int	6
3	Human Resources	7
3.1	Overview	7
3.2	humanresources.h Detail	7
3.2.1	ExperienceType	7
3.2.2	Experience	7
3.2.3	Application	8
3.2.4	Global Variables	8
3.3	Array Manipulation	8
3.3.1	Insertion	8
3.3.2	Removal	8
3.4	Functions	9

3.4.1	Add Application	9
3.4.2	Calculate Rating	9
3.4.3	Is Overqualified	10
3.4.4	Make Decision	10
3.4.5	Add Experience	10
3.4.6	Find Duplicates	11
3.4.7	Give Referral	11
3.4.8	Mass Layoffs	11
3.4.9	Search By Company	11
3.4.10	Search By Resume Gap	11
3.4.11	Sort By Rating	12
3.4.12	Print Application	12
4	Building & Testing	13
4.1	Helpful Info	13
4.1.1	Man Pages	13
4.1.2	Debugging with GDB and printf	13
4.1.3	Makefiles	13
5	Local Autograders	14
6	Debugging	15
7	Deliverables	15
8	Rules and Regulations	16
8.1	General Rules	16
8.2	Submission Conventions	16
8.3	Submission Guidelines	16
8.4	Coding Guidelines C/C++ code	16
8.5	Syllabus Excerpt on Academic Misconduct	17

1 General

1.1 Introduction

Subject: ACTION REQUIRED: Project 4 - Talent Acquisition & Restructuring

Dear students,

Welcome to Project 4. In this project, the 2110 HR team needs your help with managing the organization.

Please read through the entire document before starting. Often times we elaborate on

instructions later on in the document. Start early and if you get stuck, there's always Piazza and office hours.

Sincerely,
2110 HR Team

1.2 What functions can I use?

1. You **are** allowed to make your own helper functions in the `assembly.c` and `mansion.c` project files if you so desire.
2. You can use the following library functions in your project: **`printf`, `strlen`, `str[n]cpy`, `str[n]cmp`**.
3. Do **NOT** modify or add your own include statements. Including a new header file is NOT allowed and the autograder will get mad at you. This includes **ANY** standard library headers not already provided.
4. In lecture, you will eventually learn about dynamic memory allocation with **`malloc`**. Please note - **You do NOT need to use malloc or any dynamic memory functions in this project**. If you attempt to use malloc, the compiler will tell you to include `stdlib.h` which is **NOT** allowed.

2 Assembly Functions

Your first task for this project will be to create your assembly functions in C! Unfortunately, Unfortunately, the HR team never took CS2110, and they don't know LC-3 Assembly, so you're all on your own for this part.

Your goal is to implement all of the functions outlined below.

2.1 Overview

For this portion of the project, you will write the same functions you wrote in assembly in project 3 in C! Function headers and comments are provided for you in the `assembly.c` file. **You do not need to edit the `assembly.h` file.**

Details for each function are included below, in case you need a reminder. Remember also that you have pseudocode for all of these functions in the Project 3 PDF (although you don't have to follow it, there might be better ways with things C provides you).

2.2 Functions

You will implement the following Assembly functions in the `assembly.c` file.

2.2.1 Longest Wordle Streak

```
int longestWordleStreak(int array[], int length);
```

This function takes an array of 1s and 0s, and you will count the longest streak of consecutive 1s. All array values will either be 0 or 1.

Example:

```
array = [1,0,1,1,1,1,0,0,1]
longestWordleStreak(array) = 4
```

Please write your code in `assembly.c`. Invalid/null arguments should return `FAILURE`.

2.2.2 Valid Sudoku Row

```
int validSudokuRow(char row[]);
```

This function will give you a row of 9 characters, a sudoku row. You should check if it is a valid row, which means there are no duplicates. Return `SUCCESS` if there are no duplicates, otherwise return `FAILURE`.

- $1 \leq \text{arr}[i] \leq 9$
- `len(ARRAY) = 9`

Example:

```
array = [1, 2, 3, 7, 9, 8, 7, 5, 6]
validSudokuRow(array) = FAILURE
```

Please write your code in `assembly.c`. Invalid/null arguments should return `FAILURE`.

2.2.3 Create Letter Boxed

```
void createLetterBoxed(char* strings[], int length);
```

Given an array of strings, modifies the starts of each string (excluding the first) to form a chain where the first letter of a string is the same as the last letter of the previous string.

Example:

```
strings = ["HEY", "HELLO", "GOODBYE"]  
createLetterboxed(strings) -> strings = ["HEY", "YELLO", "OODBYE"]
```

Please write your code in `assembly.c`. Invalid/null arguments should return `FAILURE`.

2.2.4 Wordle Score

```
int wordleScore(char guess[], char solution[]);
```

This function takes in an input word of a guess, and scores it against the solution word. If the word is five letters, score the letter according to the following rules: 1. if a letter is in the word and in the correct spot, it counts for 10 points, 2. if the letter is in the word and in the wrong spot, it counts for 3 points. Note that if a letter occurs once in the word, it can only counts once in the score, even if two of it occur in the user's input. If the user input is not 5 letters, return `FAILURE`.

Example:

```
solution = "hello"  
guess = "olleh"  
wordleScore(guess, solution) -> 22
```

Please write your code in `assembly.c`. Invalid/null arguments should return `FAILURE`.

2.2.5 Valid Spelling Bee

```
int validSpellingBee(char word[], char alphabet[], char centerChar);
```

Given a word, determine whether it is a valid spelling bee word. A valid spelling be word must contain the given center character, and can only consist of letters in the given alphabet.

- The alphabet will contain only uppercase alphabetical characters (A-Z)

Example:

```
alphabet = ["A", "C", "E", "F", "Z"]  
centerChar = "C"  
word = "FACE"  
validSpellingBee(word, alphabet, centerChar) -> SUCCESS
```

Please write your code in `assembly.c`. Invalid/null arguments should return `FAILURE`.

2.2.6 Octal String to Int

```
int octalStringToInt(char octalStr[], int length);
```

This function takes in a string that represents a number in unsigned octal format, as well as its length. Convert the octal number to its decimal value.

- $'0' \leq \text{octal}[i] \leq '7'$ for all $0 \leq i < \text{length}$.
- $1 \leq \text{length} \leq 6$.
- You do not need to worry about negative numbers; you may treat all strings as unsigned binary.

Example:

```
octalString = "6767"  
octalStringToInt(octalString, 4) -> 3575
```

Please write your code in `assembly.c`. Invalid/null arguments should return **FAILURE**.

3 Human Resources

For this portion of the project, you are tasked with helping the HR team optimize the company's workforce with your C skills! To do this, the CEO has tasked you to write the bodies of the functions used to hire top-tier talent and organize the application pool to save the company's bottom line.

3.1 Overview

All of the code you write for this portion of the project will be in the `humanresources.c` file. **You do not need to edit the `humanresources.h` file.** This file will be used as a reference for the structs you will use in the `mansion.c` file.

Make sure you understand the structs in the `humanresources.h` file before continuing to fill out the bodies of the functions. Also, it is suggested you read through all of the functions before starting, as some later functions will help you make earlier ones.

3.2 `humanresources.h` Detail

This section tells you everything you need to know about the structs defined in the `humanresources.h` file.

3.2.1 `ExperienceType`

```
enum ExperienceType {
    INTERN,
    ENGINEER_JUNIOR,
    ENGINEER_SENIOR,
    MANAGER,
    MANAGER_SENIOR,
    CEO
};
```

The `ExperienceType` Enum holds the various levels in the corporate hierarchy. Each experience has exactly one type.

3.2.2 `Experience`

```
struct Experience {
    char company_name[MAX_COMPANY_LEN];
    int start_year;
    int end_year;
    enum ExperienceType experience_type;
};
```

The `Experience` struct holds all of the attributes for a single past job.

- `company_name` represents the name of the company.
- `start_year` and `end_year` define the timeline of the job. If the applicant currently works there, `end_year` will be 2026.
- `experience_type` is the enum classification of the role.

3.2.3 Application

```
struct Application {
    char first_name[MAX_APPLICANT_NAME_LEN];
    char last_name[MAX_APPLICANT_NAME_LEN];
    int age;
    struct Experience experiences[MAX_EXPER_LEN];
    int project_count;
    int has_referral;
};
```

The `Application` struct holds all of the details of an application to a job, essentially an applicant's resume.

- `first_name` and `last_name` represent the applicant's name.
- `age` is the applicant's age.
- `experiences` is an array of the applicant's past job experiences.
- `project_count` represents the number of projects they have completed.
- `has_referral` is a boolean flag (1 or 0) indicating if they know someone inside the company.

3.2.4 Global Variables

`applications` is a global array that represents all the current applications in the HR system. This array must remain contiguous, so no gaps should be present between array entries. There can be a max of `MAX_APPLICATIONS_LEN` applications.

`app_count` is a global integer that represents how many applications are currently in the system, cannot go over the maximum number of applications.

3.3 Array Manipulation

For any fixed-length arrays (i.e. `applications`), you **must** follow certain policies for insertion and removal to comply with what the autograder expects. If you do not, it may be difficult to debug test cases.

3.3.1 Insertion

When inserting into or adding to an array, you **must** insert at the end of the array. This can be done with the `app_count` variable that represents the number of elements in the array as the index to insert at.

Example: `[6, 12, -4, 5]` (insert 8) $\rightarrow$ `[6, 12, -4, 5, 8]`

3.3.2 Removal

When removing or deleting from an array, after you remove, you **must** copy the last valid element to the index of the removed element. This can also use the `app_count` variable for calculating the last valid index.

Example: `[6, 12, -4, 5]` (remove 12) $\rightarrow$ `[6, 5, -4]`

Additionally, when removing elements in a loop, you **must** remove the elements in reverse order; otherwise, the elements will not be removed correctly.

For example, suppose we wanted to remove all even numbers from `[1, 2, 3, 4, 5, 6]`. If we tried to iterate in forward order, we will see this removal process occur (notice that not every even number has been removed):

```

      v
[1, 2, 3, 4, 5, 6]
      v
[1, 2, 3, 4, 5, 6]
      v
[1, 6, 3, 4, 5]
      v
[1, 6, 3, 5]

```

You **must** iterate in reverse order (notice that every even number has been removed):

```

      v
[1, 2, 3, 4, 5, 6]
      v
[1, 2, 3, 4, 5]
      v
[1, 2, 3, 4, 5]
      v
[1, 6, 3, 5]
      v
[1, 6, 3, 5]
      v
[1, 5, 3]

```

3.4 Functions

The following are the details for the functions the HR team needs you to implement.

3.4.1 Add Application

```

int addApplication(const char *first_name, const char *last_name,
                  int age, int project_count, int has_referral);

```

This function adds an application with the given attributes to the array of applications. All applications will also have experiences, but this array should be initialized as an empty array when adding the application.

This function should return **ERROR** if any of the following conditions are true:

- The name is too long.
- The name is empty.
- The age or project count is invalid (cannot be negative)
- The referral value is not 0 or 1
- Any of the inputs are null

Otherwise, add the application to the system, which is the applications array, and return **SUCCESS**.

3.4.2 Calculate Rating

```

int calculateRating(struct Application *app);

```

This function calculates a rating for each application based on its features. The formula for the rating is as follows: **Experience Value + Project Value + Referral Value.**

- **Experience value** is calculated with (position weight * years worked), summed across all experience values.
- Years worked is (end_year - start_year). If it calculates to 0, then it should be treated as 1 year
- Position weight should be taken from the experience struct field
- Each project adds 5 points to the rating, but 0 projects subtracts 5 points from the rating.
- If the applicant has a referral, add 10 points to the rating.

Make sure that there is no negative rating; if the final rating is negative, return 0. If the application pointer is NULL, return ERROR.

3.4.3 Is Overqualified

```
int isOverqualified(struct Application *app, enum ExperienceType role);
```

This function determines if an applicant is overqualified for a given role. An applicant is overqualified if their **most recent** experience position is higher than the given role. For example, if the role is **MANAGER**, the most recent experience being an **INTERN** or **MANAGER** is not overqualified, but a **CEO** is overqualified.

Note that the experiences array is not guaranteed to be in order, so you must iterate through to find the most recent experience based on start_year.

Returns **SUCCESS** if the applicant is overqualified, **FAILURE** if they are not, and **ERROR** if the input is invalid.

3.4.4 Make Decision

```
int makeDecision(struct Application *app, int min_rating, enum ExperienceType role);
```

This function determines whether an applicant can be accepted based on their resume.

- If the applicant's rating is below the given minimum rating, reject them.
- If the applicant is overqualified for the given role, reject them.
- Otherwise, accept them.

Returns **SUCCESS** if the applicant is accepted, **FAILURE** if they are not, and **ERROR** if the input is invalid.

3.4.5 Add Experience

```
int addExperience(const char *first_name, const char *last_name,  
                 const char *company_name, int start_year, int end_year,  
                 enum ExperienceType experience_type);
```

This function will add an experience to an application. Given the applicant's name, find the application and add to their experience.

Must check for edge cases to make sure the experience is valid. Think about what would make a job experience invalid, and what would make other calculations in this project be incorrect.

Returns **SUCCESS** if the experience was added successfully, **FAILURE** if no such application exists, and **ERROR** if the experience is invalid.

3.4.6 Find Duplicates

```
int findDuplicates(void);
```

This function checks for duplicate applications in the `applications` array. Two applications are considered duplicates if they have the exact same first name, last name, and age.

Returns `SUCCESS` if there are duplicates, and `FAILURE` if there are no duplicates.

3.4.7 Give Referral

```
int giveReferral(const char *first_name, const char *last_name);
```

This function is intended to update an application to give them a referral. Must check for the application, and set the referral value to 1.

Returns `SUCCESS` if the referral was set successfully, `FAILURE` if no such application exists, and `ERROR` if the name is invalid.

3.4.8 Mass Layoffs

```
int massLayoffs(void);
```

This function is intended to update the applicant pool by removing the “weaker” applications. Must remove applications that meet any of the following criteria:

- Applicants whose most recent experience is an `INTERN`.
- Applicants who have less than 3 projects.
- Applicants whose first name starts with a ‘J’ or ‘T’.

After removing these applications, make sure the array has no gaps. Refer to section 3.3 for help on how to remove from the array without having any gaps between applications.

3.4.9 Search By Company

```
int searchByCompany(const char *company, char results[][MAX_APPLICANT_NAME_LEN * 2]);
```

This function searches for applicants who have worked at a given company and adds their names to the `results` array.

Note that an applicant may have multiple experiences at the same company, but they should only be added to the results array once. Format the strings as “FirstName LastName”. Make sure there is a space between.

Returns the number of applicants who worked at the company, or `ERROR` if the input is invalid.

3.4.10 Search By Resume Gap

```
int searchByResumeGap(char results[][MAX_APPLICANT_NAME_LEN * 2]);
```

This function searches for applicants who have had a resume gap and adds their names to the `results` array. A resume gap is defined as any difference of 3 or more years between consecutive experiences.

Note that an applicant may have multiple resume gaps, but they should only be added to the results array once. Format the strings as “FirstName LastName”. Make sure there is a space between.

Returns the number of applicants who have resume gaps, or `ERROR` if the input is invalid.

3.4.11 Sort By Rating

```
int sortByRating(void);
```

This function sorts the `applications` array. You should sort in descending order by rating, so the highest-rated application is at the front of the array (index 0). If two applications have the same rating, sort them in descending order by project count.

You may use any sorting algorithm you like. Returns `SUCCESS` upon completion.

3.4.12 Print Application

```
void printApplication(struct Application *app);
```

This function prints out the details of a single application. For simplicity, you can print the applicant's name, age, and project count. You are free to format this however you like. Get creative! This is to make sure you understand how to use `printf` statements. (*Note: This function will not be strictly autograded*).

4 Building & Testing

4.1 Helpful Info

4.1.1 Man Pages

The `man` command in Linux provides “an interface to the on-line reference manuals.” If you are unsure about the specifics of a `my_string.c` function, you should look up the exact details using its man page. This is a great utility for any C and Linux developer for finding out more information about the available functions and libraries. In order to use this, you just need to pass in the function name to this command within a Linux (in our case Docker) terminal.

For instance, entering the following command will print the corresponding man page for the `strlen` function:

```
$ man strlen
```

Additionally, the man pages are accessible online at: <http://man.he.net>

NOTE: You can ignore the subsections after the “RETURN VALUE” (such as ATTRIBUTES, etc) for this homework. However, pay close attention to function descriptions.

4.1.2 Debugging with GDB and printf

We highly recommend getting used to “`printf` debugging” in C early on.

Moreover, If you run into a problem when working on your homework, you can use the debugging tool, GDB, to debug your code! Former TA Adam Suskin made a series of tutorial videos which you can find [here](#). GDB is an essential tool for any C program, so we definitely recommend learning it now.

When running GDB, if you get to a point where user input is needed, you can supply it just like you normally would. When an error happens, you can get a Java-esque stacktrace using the `backtrace` (`bt`) command which allows you to pinpoint where the error is coming from. For more info on basic GDB commands, search up “GDB Cheat Sheet”.

4.1.3 Makefiles

Make is a common build tool for abstracting the complexity of working with compilers directly. Makefiles let you define a set of desired targets (files you want to compile), their prerequisites (files which are needed to compile the target), and sets of directives (commands such as `gcc`, `gdb`, etc.) to build those targets. In all of our C assignments (and also in production level C projects), a Makefile is used to compile C programs with a long list of compiler flags that control things like how much to optimize the code, whether to create debugging information for `gdb`, and what errors we want to show. We have already provided you a Makefile for this homework, but we highly recommend that you take a look at this file and understand the `gcc` commands and flags used to understand how to compile C programs. If you’re interested, you can also find more information regarding Makefiles [here](#).

Since your program is connected to an autograder with multiple files that need to be compiled using particular settings, it’s a little difficult to compile it by hand. The Makefile allows us to simply type the command `make` followed by a target such as `tests`, `run-tests`, or `run-gdb` to compile and run your code.

Please **DO NOT** modify your makefile as it may break your local autograder.

5 Local Autograders

We have provided a utility shell script `./grade.sh` (or `.\grade.bat`) which you can use to run the local autograder. These test cases are equivalent to the ones on Gradescope and you may use Gradescope instead. (However, the local autograder is faster and more configurable!)

Note that the local autograder requires Docker to run - but do not manually open the docker file! The `grade.sh` or `grade.bat` files will do that for you.

Once you have Docker Desktop open and you are `cd`'d into your project directory, you can run any of the following command `./grade.sh` on macOS/Linux or `.\grade.bat` on Windows.

If you want to debug your code more in depth, you can use the `make` commands you've used previously within the Docker container.

First, open the Docker container with `./cs2110docker.sh` (macOS, Linux) or `.\cs2110docker.bat` (Windows). Once you're in the container, you can run the following:

- `make TEST=test_name`: Run one specific test
- `make run-gdb TEST=test_name`: Run one specific test through GDB
- `make driver && ./driver`: Compile your code (with `main.c` as the entrypoint) and run the `main` function defined in `main.c`.

Note that `make TEST=test_name` is identical to `./grade.sh c test_name`. You can use whichever you prefer!

6 Debugging

Remember the local autograder is provided for you so that you can debug your code! Please look at the test case failures to make sure you understand how to work through your code!

Here is an example of a C test case:

```
[----] suites/test_c.c:16: Assertion Failed
[----]
[----]   Expected to foobar correctly
[----]
[----]   eq(ptr, result2, p2):
[----]     Actual: 0x0
[----]     Expected: 0xaaaabfcc4b50
[FAIL] test_foobar::basic: (0.00s)
```

This test case failure similarly will provide you some important information!

- The expected and actual values, located at the bottom of the error (0x0 and 0xaaaabfcc4b50 respectively).
- The test case you failed (`test_foobar::basic`).
- What line you failed (`suites/test_c.c:16`).

If you are getting totally stuck on how to debug the problem, please don't be afraid to look at the provided test cases! Look at line 16! Read what the test case is testing for!

Important Notes:

1. Only the `.c` files will be graded on Gradescope.
2. All non-compiling homework will receive a zero (with all the flags specified in the Makefile/Syllabus).
3. **NOTE: DO NOT MODIFY THE HEADER FILES.**

Since you are not turning them in, any changes to the `.h` files will not be reflected when running the Gradescope autograder.

We reserve the right to update the autograder and the test case weights on Gradescope or the local checker as we see fit when grading your solution.

7 Deliverables

Turn in `assembly.c` and `humanresources.c` to Gradescope by the due date.

Please do not wait until the last minute to run/test your project, history has proved that last minute turn-ins will result in long queue times for grading on Gradescope. You have been warned!!!

8 Rules and Regulations

8.1 General Rules

1. Although you may ask TAs for clarification, you are ultimately responsible for what you submit.
2. Please read the assignment in its entirety before asking questions.
3. Please start assignments early, and ask for help early. Do not email us a few hours before the assignment is due with questions.
4. If you find any problems with the assignment, it would be greatly appreciated if you reported them to the author (which can be found at the top of the assignment). Announcements will be posted if the assignment changes.

8.2 Submission Conventions

1. In order to submit your assignment, submit the files individually to the Gradescope assignment.
2. Do not submit links to files. The autograder does not understand it, and we will not manually grade assignments submitted this way as it is easy to change the files after the submission period ends.

8.3 Submission Guidelines

1. You are responsible for turning in assignments on time. This includes allowing for unforeseen circumstances. If you have an emergency let us know **IN ADVANCE** of the due time supplying documentation (i.e. note from the dean, doctor's note, etc). Extensions will only be granted to those who contact us in advance of the deadline and no extensions will be made after the due date.
2. You are also responsible for ensuring that what you turned in is what you meant to turn in. After submitting you should be sure to download your submission into a brand new folder and test if it works. No excuses if you submit the wrong files, what you turn in is what we grade. In addition, your assignment must be turned in via Gradescope. Under no circumstances whatsoever we will accept any email submission of an assignment. Note: if you were granted an extension you will still turn in the assignment over Gradescope.
3. Projects turned in late receive partial credit within the first 48 hours, as defined in the syllabus. Between 0 and 24 hours late, you can receive a maximum score of 70%. Between 24 and 48 hours late, you can receive a maximum score of 50%. We will not accept projects turned in over 48 hours late.
4. You alone are responsible for submitting your project before the assignment is due; neither Canvas/Gradescope, nor your flaky internet are to blame if you are unable to submit because you banked on your computer working up until the deadline.

8.4 Coding Guidelines C/C++ code

1. You must turn in ALL files specified in the "Deliverables" section of the assignment instructions. We reserve the right to impose a penalty on submissions that do not follow the given submission directions.
2. You must provide a Makefile that compiles and links your code by default. If you are given a Makefile with the project, we expect your code to compile with **make**.
3. Your code must compile with gcc on Ubuntu 22.04 LTS (aka compile using the CS 2110 Docker container). If your code does not compile, you will receive a 0 for the assignment.

4. You will be penalized if your code produces warnings when compiled with the given Makefile, or the following flags if no Makefile is provided: `gcc -Wall -Wextra -Wstrict-prototypes -pedantic -O2`
5. Code should be well commented and use a clean, consistent (readable) style (i.e., proper indenting, etc.). We reserve the right to impose style requirements, and deduct for non-conforming solutions. This is not the obfuscated C code competition (<http://ioccc.org>).

8.5 Syllabus Excerpt on Academic Misconduct

The goal of all assignments in CS 2110 is for you to learn. Learning requires thought and hard work. Copying answers thus prevents learning. More importantly, it gives an unfair advantage over students who do the work and follow the rules.

1. **As a Georgia Tech student, you have read and agreed to the Georgia Tech Academic Honor Code.** The Academic Honor Code defines Academic Misconduct as “*any act that does or could improperly distort Student grades or other Student academic records.*”
2. You must submit an assignment or project as your own work. **No collaboration on answers is permitted. Absolutely no code or answers may be copied from others. Such copying is Academic Misconduct.**
3. Using code from GitHub, via Googling, from Stack Overflow, etc., is Academic Misconduct (Honor Code: Academic Misconduct includes “*submission of material that is wholly or substantially identical to that created or published by another person or persons*”).
4. Publishing your assignments on public repositories, github, etc, that is accessible to other students is unauthorized collaboration and thus Academic Misconduct.
5. Suspected Academic Misconduct will be reported to the Division of Student Life Office of Student Integrity. It will be prosecuted to the full extent of Institute policies.
6. Students suspected of Academic Misconduct are informed **at the end of the semester**. Suspects receive an *Incomplete* final grade until the issue is resolved.
7. We use accepted forensic techniques to determine whether there is copying of a coding assignment.
8. Submitting an assignment with code or text from an AI assistant (e.g., ChatGPT) is academic misconduct.
9. **If you are not sure about any aspect of this policy, please ask your lecturer.**

Using AI Assistants

Anything you did not write in your assignment will be treated as an academic misconduct case. If you are unsure where the line is between collaborating with AI and copying AI, we recommend the following heuristics:

Heuristic 1: Never hit “Copy” within your conversation with an AI assistant. You can copy your own work into your own conversation, but do not copy anything from the conversation back into your assignment. Instead, use your interaction with the AI assistant as a learning experience, then let your assignment reflect your improved understanding.

Heuristic 2: Do not have your assignment and the AI agent open at the same time. Similar to the above, use your conversation with the AI as a learning experience, then close the interaction down,

open your assignment, and let your assignment reflect your revised knowledge. This heuristic includes avoiding using AI directly integrated into your composition environment: just as you should not let a classmate write content or code directly into your submission, so also you should avoid using tools that directly add content to your submission.

Deviating from these heuristics does not automatically qualify as academic misconduct; however, following these heuristics essentially guarantees your collaboration will not cross the line into misconduct.